

Arquitetura Neuro-Simbólica para Manutenção Prescritiva em Borda: Integração de Modelos de Linguagem e Diagnóstico Numérico

O Paradigma da Computação de Borda e o Imperativo Neuro-Simbólico

A convergência entre a inteligência artificial generativa e os ambientes industriais impõe desafios arquiteturais singulares, particularmente quando submetida a restrições estritas de infraestrutura. O desenho de um pipeline completo de Manutenção Prescritiva para máquinas rotativas, destinado a operar integralmente numa estação de trabalho comercial singular com recursos finitos (16 GB de VRAM e 32 GB de RAM), exige um afastamento radical das abordagens convencionais baseadas em computação na cloud. A proibição do uso de Interfaces de Programação de Aplicações (APIs) externas, como as fornecidas pela OpenAI ou Anthropic, dita a necessidade de uma solução *on-premise* robusta, capaz de operar em ambientes de rede isolados, garantindo a soberania dos dados operacionais e a mitigação da latência de inferência¹.

O objetivo central deste sistema é duplo e sequencial: primeiro, diagnosticar o defeito mais provável através da busca de similaridade matemática numa base de dados histórica de eventos vibracionais; segundo, utilizar um Modelo de Linguagem de Grande Escala (LLM) para, a partir deste diagnóstico exato, orquestrar a Geração Aumentada por Recuperação (RAG) sobre uma base de documentos técnicos e gerar uma resposta prescritiva para o operador. A literatura acadêmica e as melhores práticas da indústria concordam que a utilização de LLMs para a inferência direta de dados numéricos brutos — tais como tolerâncias mecânicas lineares ou limiares de vibração — constitui uma antiprática². As redes neurais generativas são inerentemente probabilísticas e propensas à alucinação analítica, carecendo do rigor determinístico necessário para cálculos algébricos exatos⁴. Para suplantar esta limitação sistêmica, a arquitetura proposta adota um design neuro-simbólico. Este paradigma segrega as responsabilidades de processamento: um "Motor Simbólico" baseado em CPU executa a avaliação estatística rigorosa dos dados dos sensores, enquanto um "Motor Neural" confinado à GPU assume exclusivamente o processamento semântico e a interação em linguagem natural com o utilizador⁵. A orquestração destes dois domínios é regida por um framework agêntico de grafos, assegurando um fluxo de dados resiliente, controlável e imune a invenções de diagnóstico.

Orçamento de Memória e Seleção de Modelos

Fundacionais

A restrição primária de hardware, consubstanciada num máximo de 16 GB de VRAM dedicada e 32 GB de memória de sistema (RAM), atua como o vetor determinante para a seleção tecnológica de toda a camada neural. Modelos fundacionais operando em precisão original de 16 bits (FP16 ou BF16) exigem capacidades de memória que excedem largamente este limite, tornando a quantização de pesos e a otimização da gestão de contexto passos obrigatórios para a viabilidade do projeto¹.

Avaliação e Seleção do Modelo de Linguagem (LLM)

Para a tarefa de prescrição técnica e raciocínio sobre documentos de engenharia mecânica, o modelo deve possuir alta capacidade lógica, alinhamento rigoroso a instruções e um tamanho de parâmetros que, após compressão matemática, resida confortavelmente na VRAM. Uma análise exaustiva das opções de código aberto revela duas principais alternativas para este limiar de hardware: o Llama 3.1 8B e o Qwen 2.5 14B⁷.

O Llama 3.1 8B Instruct, quantizado no formato GGUF com a especificação Q4_K_M (4 bits), consome aproximadamente 5,5 GB de VRAM para o armazenamento dos pesos estruturais⁷. Ao prever um contexto de 16.000 tokens — necessário para o processamento de múltiplos fragmentos de manuais técnicos recuperados pelo sistema RAG — o cache de Chaves e Valores (KV Cache) em precisão FP16 adicionará cerca de 2,3 GB, resultando num consumo total na ordem dos 8,8 GB¹¹. Embora esta configuração deixe uma ampla margem de VRAM livre, ela subutiliza o hardware disponível, alocando um modelo de apenas 8 bilhões de parâmetros quando a infraestrutura permite maior densidade de conhecimento.

Por conseguinte, a arquitetura selecionada recai sobre o modelo Qwen 2.5 14B Instruct⁷. Avaliações de desempenho independentes e benchmarks demonstram que o Qwen 2.5 14B supera significativamente modelos da classe de 8B em tarefas de raciocínio, geração de código e, criticamente, na interpretação de linguagem estruturada e conformidade com esquemas JSON¹⁰.

Table: Comparação de Modelos Candidatos para 16 GB de VRAM (Quantização Q4_K_M)

Métrica / Parâmetro	Llama 3.1 8B Instruct	Qwen 2.5 14B Instruct	Mistral 7B v0.3
Parâmetros	8,0 Bilhões	14,7 Bilhões	7,2 Bilhões
Tamanho dos Pesos (VRAM)	~5,5 GB ¹¹	~8,7 a 9,0 GB ⁷	~4,4 GB ⁷
KV Cache (8K tokens)	~1,1 GB ⁷	~2,0 GB ⁷	~1,0 GB ⁷

KV Cache (16K tokens)	~2,3 GB ¹¹	~4,0 GB	~2,0 GB
Consumo Total Estimado (16K)	~8,8 GB ¹¹	~13,0 GB	~7,4 GB
Desempenho em Raciocínio (MMLU)	69,4% ¹⁰	79,7% ¹⁰	Inferior ao Llama 3.1

Quantizado no formato Q4_K_M, o Qwen 2.5 14B exige cerca de 9,0 GB de VRAM para a inferência base⁸. Ao alocar um contexto operacional de 8.000 a 12.000 tokens para o pipeline de RAG, o KV Cache consumirá entre 2 GB e 3 GB¹. Esta alocação perfaz um total de aproximadamente 12 GB, deixando 4 GB de VRAM integralmente livres. Esta margem de segurança é vital para acomodar o sistema operativo, sobrecargas de alocação CUDA (buffers de contexto), e eventuais flutuações de memória durante picos de geração de tokens⁷.

O Motor de Inferência Local: Llama.cpp vs. vLLM

A escolha do motor de inferência dita a eficiência com que o modelo gere a memória, sendo tão crítica quanto a seleção dos pesos neuronais. Motores desenhados para ambientes de produção massiva na cloud, como o vLLM, alavancam a técnica de *PagedAttention* para maximizar o processamento simultâneo de requisições em lote¹⁵. Não obstante, o vLLM adota uma política de alocação de memória extremamente agressiva, reservando preemptivamente 90% da VRAM total da placa gráfica (através do parâmetro `--gpu-memory-utilization 0.9`) para assegurar espaço para o pool de páginas¹. Em arquiteturas restritas a 16 GB, esta reserva resulta numa falência imediata ao tentar carregar modelos de 14B, induzindo erros de falta de memória (Out-Of-Memory - OOM)¹.

Adicionalmente, o vLLM aloca cache KV para todas as camadas do modelo, adicionando sobrecargas de preenchimento (padding) e espaço de trabalho CUDA que podem inflacionar o uso de memória em 50% além do estritamente necessário para um único utilizador¹⁷.

Para este cenário de *edge computing*, onde a estabilidade de uma sessão única com contexto longo suplanta a necessidade de concorrência massiva, o llama.cpp é a escolha arquitetural imperativa. O llama.cpp opera com uma alocação de KV Cache cirúrgica, limitando-se a aproximadamente 16 KB por token apenas para as camadas de atenção, e optando por recalcular as redes neurais *feed-forward* (FFNs) para economizar VRAM¹⁷. Esta gestão otimizada garante que contextos de até 16.000 tokens caibam com segurança nos 4 GB remanescentes.

Para estabilizar a execução numa máquina permanentemente ligada, recomenda-se servir o llama.cpp como um binário compilado nativamente com suporte CUDA (através de bibliotecas Python como llama-cpp-python). Parâmetros de execução como `--mlock` devem ser ativados para forçar o sistema operativo a manter o modelo estritamente na memória, evitando paginações indesejadas que destruiriam o rendimento temporal (tokens por segundo)¹⁶. A

desativação do mapeamento de memória em disco (`--no-mmap`) também é sugerida para instâncias em que os pesos cabem integralmente na VRAM, forçando uma carga residente completa¹⁴.

O Motor de Embeddings Espaciais

Para preservar a GPU exclusivamente para o LLM, o processo de criação de *embeddings* — a vetorização geométrica dos documentos e das consultas para a busca semântica — deve ser alocado integralmente à CPU, tirando partido da folga nos 32 GB de RAM.

A utilização da biblioteca FastEmbed é a solução mais eficaz para este designio²⁰. O FastEmbed foi concebido com o propósito de eliminar dependências volumosas como o PyTorch, operando diretamente sobre o ONNX Runtime. Isto permite que os modelos de *embedding* tirem partido de otimizações ao nível do silício do processador, como as instruções AVX2 e AVX512, garantindo inferências hiper-rápidas sem sobrecarregar a placa gráfica²². O modelo de *embedding* eleito é o BAAI/bge-m3, amplamente reconhecido pela sua precisão no topo das tabelas de classificação MTEB e pela sua capacidade inerente de suporte multilingue e de arquitetura de múltiplos vetores (denso, esparso e ColBERT)²⁵. Quando quantizado em formato INT8 e executado via FastEmbed, o modelo exige apenas cerca de 1,5 GB a 2 GB de RAM, suportando paralelização nativa em múltiplos núcleos lógicos do processador (parâmetro `parallel`) e carregamento sob demanda (`lazy_load`)²⁰. Esta separação física dos pipelines evita a contenção na interface PCIe e bloqueios cruzados entre a criação de vetores e a geração de texto²⁰.

O Motor Simbólico: Avaliação de Severidade e Diagnóstico Numérico

O cerne do rigor neuro-simbólico reside no tratamento da componente numérica. O sistema recebe um objeto JSON contendo métricas estatísticas extraídas de sensores de vibração. A interpretação destes dados não é um exercício de linguagem, mas sim de mecânica das vibrações, regido por normativas globais de fiabilidade.

Fundamentação Física das Métricas de Vibração

Os atributos capturados pela engenharia de *features* são reflexos diretos da degradação física dos equipamentos, com base nas diretrizes da norma internacional ISO 20816-3 (que sucedeu à ISO 10816-3). Esta norma estabelece o enquadramento imperativo para a avaliação da severidade vibracional em maquinário rotativo acoplado com potência superior a 15 kW e velocidades entre 120 rpm e 30.000 rpm².

A arquitetura simbólica interpreta as seguintes métricas fundamentais:

Table: Significado Físico das Features Estatísticas de Vibração

Feature Recebida (JSON)	Significado Físico e Utilidade Diagnóstica
-------------------------	--

Velocidade RMS (mm/s)	Métrica de banda larga (frequentemente entre 10 e 1000 Hz) que representa a energia destrutiva global da máquina. É o parâmetro de base da ISO 20816-3 para categorizar o estado da máquina em Zonas (A, B, C, D) e diagnosticar desbalanceamento ou desalinhamento ²⁹ .
Pico de Aceleração	Valor de crista no espectro de aceleração, vital para diagnosticar defeitos incipientes em altas frequências. Revela forças de impacto metálico transiente antes destas gerarem energia suficiente para alterar o valor RMS global ²⁹ .
Fator de Crista	O rácio entre a amplitude de pico e o valor RMS de um sinal. Um Fator de Crista elevado é o indicador mais precoce de falhas em rolamentos; indica impactos de alta energia (picos) numa altura em que o desgaste contínuo (RMS) ainda é diminuto ²⁹ .
Curtose (Kurtosis)	O quarto momento estatístico, que mede a excentricidade das caudas da distribuição de probabilidade do sinal no domínio do tempo. Uma máquina saudável exibe uma distribuição Gaussiana normal com Curtose ~3.0. Valores acima de 4.0 a 5.0 evidenciam a presença de picos impulsivos esporádicos, característicos de fragmentação nas pistas de rolamentos ³⁴ .
Frequências de Defeito (BPFO, BPFI, BSF, FTF)	Frequências teóricas calculadas pela geometria do rolamento (Pista Externa, Pista Interna, Frequência de Rotação da Esfera, e Frequência Fundamental do Trem/Gaiola). Picos exatos nestes multiplicadores isolam o defeito a componentes específicos ³⁵ .

A classificação de severidade aplica limites que dependem do Grupo da Máquina e da rigidez do seu suporte (fundação rígida ou flexível). Por exemplo, para uma máquina média (Grupo 2,

de 15 kW a 300 kW) assente sobre suportes rígidos, a transição para a Zona D (Perigo - Risco de Danos) ocorre quando a velocidade RMS ultrapassa os 4,5 mm/s³⁰.

Distância de Mahalanobis e Detecção Fora de Distribuição (OOD)

O JSON recebido é vetorizado numa matriz de *features*. O objetivo primário é encontrar a anomalia correspondente na base de dados histórica (banner.csv). A mera aplicação da distância Euclidiana ou da proximidade do co-seno seria um erro metodológico primário. As variáveis vibracionais (ex.: RMS em milímetros por segundo e Curtose como um escalar sem dimensão) não partilham escalas isométricas e apresentam forte covariância. Por exemplo, o agravamento de um desbalanceamento aumenta simultaneamente o RMS e afeta as harmónicas de ordem 1X e 2X de forma correlacionada³¹.

A métrica apropriada implementada no Motor Simbólico é a Distância de Mahalanobis, acoplada a um algoritmo k-Nearest Neighbors (k-NN) com pesos³⁷. A Distância de Mahalanobis avalia a proximidade de um ponto de teste a um centroide de classe tendo em consideração a correlação multivariável entre as propriedades do conjunto de dados, modelando a distribuição estatística real³⁹.

A equação implementada na CPU via processamento matricial é:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Onde $\mathbf{x}$ é o vetor do novo sinal de vibração, $\boldsymbol{\mu}$ é o vetor médio (centroide) das assinaturas de falhas conhecidas na base de dados, e $\boldsymbol{\Sigma}^{-1}$ é a matriz de covariância inversa das características de falha³⁹. Para prevenir singularidades em dados altamente colineares, a arquitetura deve utilizar a pseudo-inversa de Moore-Penrose, recorrendo à função `scipy.linalg.pinv` do Python, assegurando estabilidade numérica perante matrizes mal condicionadas³⁹.

Esta abordagem simbólica confere duas vantagens inigualáveis à arquitetura:

1. **Diagnóstico Determinístico:** O k-NN ponderado devolve a classificação exata do defeito (ex.: "Ruptura na Pista Interna - BPFI") com base em precedentes de engenharia puramente matemáticos, sem influência semântica³⁷.
2. **Detecção de Anomalias Inéditas (Out-of-Distribution - OOD):** A distância de Mahalanobis serve como um avaliador de confiança hiper-robusto⁵. Se o valor escalar de $D_M(\mathbf{x})$ calculado exceder o limiar de confiança estatística (por exemplo, um escore-Z superior a 3 na distribuição qui-quadrado associada), o motor matemático classifica imediatamente a ocorrência como uma anomalia Fora de Distribuição (OOD)⁵. Esta sinalização é o gatilho central para a política de mitigação de alucinações na orquestração subsequente.

Orquestração Agêntica com LangGraph

Para garantir que o fluxo de informação não seja um mero encadeamento sequencial e passivo, mas um processo autoconsciente e ramificável com base na certeza estatística, introduz-se a orquestração agêntica baseada na biblioteca LangGraph. O LangGraph modela os fluxos de trabalho de agentes de inteligência artificial através do formalismo de grafos direcionais cíclicos, nos quais nós (Nodes) representam unidades de computação modular e arestas (Edges) governam as regras de transição de estados⁴³.

Definição do Esquema de Estado (State Schema)

O mecanismo de passagem de mensagens entre os nós no LangGraph assenta num dicionário de estado unificado. Este estado é definido de forma estrita através de estruturas TypedDict em Python, impondo um contrato de dados rígido que flui do motor matemático ao motor semântico⁴³.

O esquema proposto para a manutenção prescritiva inclui:

- `sensor_data`: Payload JSON limpo e normalizado.
- `identified_fault`: String contendo o defeito identificado (ou `unknown_anomaly`).
- `confidence_score`: Valor numérico (float) derivado da Distância de Mahalanobis.
- `retrieved_context`: Lista de fragmentos (chunks) documentais recuperados do motor RAG.
- `messages`: O histórico de interação entre o operador e o sistema (utilizando o padrão `MessagesAnnotation` para rastreamento conversacional contínuo)⁴⁴.

Quando um nó emite atualizações, o LangGraph utiliza funções redutoras (*reducers*) predefinidas para fundir as novas informações com o estado histórico. Para o histórico de mensagens, utiliza-se a semântica de *append* (adição), garantindo que as chamadas LLM seguintes retenham a memória das interações passadas de forma segura e não destrutiva⁴³.

Topologia de Nós e Execução por Superpassos

A execução do grafo divide-se numa sucessão de "superpassos" lógicos⁴⁴. A topologia concebida é a seguinte:

1. **Nó Recetor (Receiver Node)**: Intercepta o JSON a partir do endpoint FastAPI. Executa validação fronteiriça via Pydantic (garantindo que não existem valores nulos ou fisicamente impossíveis) e inicializa a classe de estado. Se a validação falhar, o fluxo é abortado preventivamente, poupando recursos de computação⁴⁶.
2. **Nó de Diagnóstico Simbólico (Diagnostic Node)**: Invoca a rotina K-NN com Mahalanobis sobre o pacote de dados. Atualiza os campos `identified_fault` e `confidence_score` no estado com base nos resultados.
3. **Aresta Condicional de Roteamento (Router Edge)**: Aqui reside o mecanismo de proteção neuro-simbólica fundamental. É definida uma aresta condicional (usando a função `add_conditional_edges`) imediatamente após o Nó Diagnóstico⁴⁷. Uma função heurística avalia o `confidence_score`:
 - *Caso A (Confiança Adequada)*: Se a distância for baixa (dentro da distribuição normal), a função retorna a constante literal `"rag_retrieval_node"`.
 - *Caso B (Anomalia OOD)*: Se a distância exceder o limiar predefinido (sinalizando

uma anomalia inédita, não presente no banner.csv), a função ramifica a execução retornando "novel_fault_node"⁴⁵.

4. Nós de Resolução:

- **Nó de Recuperação RAG (RAG Node):** Acionado pelo Caso A. Constrói dinamicamente uma *query* contendo a categoria do defeito, extrai os manuais orientativos da base vetorial local e anexa a literatura ao estado⁴⁶.
- **Nó de Falha Inédita (Novel Fault Node):** Acionado pelo Caso B. Ignora a extração especializada. Injeta um contexto instrucional rígido (System Prompt) no estado: *"Atenção: A assinatura estatística atual não corresponde a nenhum padrão catalogado. O utilizador deve ser alertado de que o defeito é desconhecido. Não formule soluções inventadas. Prescreva a adoção do protocolo de emergência padrão (desligar a máquina) e solicite a criação de um ticket de engenharia para inspeção física."*⁴⁹.

5. **Nó Prescritivo Generativo (Prescriber Node):** Consolida todos os dados de estado e envia um pacote formatado para a GPU, instigando o Qwen 2.5 14B a gerar a resposta final num tom adequado.

Esta orquestração garante que o Motor Neural atua unicamente como um mecanismo de leitura e síntese lógica do diagnóstico exato e da literatura pertinente, eliminando por desenho a probabilidade de o modelo tentar extrapolar explicações fantasiosas para assinaturas de vibração complexas⁴⁵.

Estratégia de RAG e Indexação Vetorial Avançada

A indexação dos manuais técnicos não pode ser executada por ferramentas de fragmentação ingênuas. Os catálogos industriais são densamente povoados por vocabulário específico, abreviações (ex.: tolerâncias ISO h6/H7) e instruções em formato tabular.

Tratamento e Fragmentação de Documentos

A estratégia imperativa para a preservação semântica é o "Chunking Hierárquico" através do método Pai-Filho (*Parent-Child Retrieval*), suportado por ferramentas de conversão como o PyMuPDF⁵¹.

A limpeza e remoção excessiva de ruído deve ser calibrada cuidadosamente para evitar a destruição de notações técnicas essenciais⁵². O particionamento é estruturado em duas camadas: o documento é dividido em nós "pais", englobando secções inteiras com a explicação holística de um procedimento; por sua vez, estes são subdivididos em múltiplos nós "filhos", fragmentos muito mais pequenos e específicos, que concentram palavras-chave altamente densas. A vetorização (processamento pelo FastEmbed) ocorre exclusivamente nos nós filhos. Contudo, quando a pesquisa encontra a proximidade matemática num nó filho, o sistema recupera e entrega o texto completo do nó pai ao contexto do LLM⁵². Isto garante uma busca com precisão cirúrgica de agulha num palheiro, combinada com uma entrega contextual vasta que permite ao Qwen 2.5 formular respostas fluidas e logicamente embasadas.

Seleção e Configuração da Base de Dados Vetorial: Qdrant

A gestão dos embeddings deve ser confiada a uma base de dados vetorial dedicada. Na análise comparativa entre as opções dominantes em código aberto — Milvus e Qdrant —, o Qdrant evidencia-se como a arquitetura superior para um cenário de *edge computing* numa estação de trabalho isolada⁵³.

Enquanto o Milvus foi desenhado como um sistema distribuído de microserviços Kubernetes para lidar com bilhões de vetores — exigindo a execução paralela de contentores auxiliares como o *etcd* e o *MinIO* —, o Qdrant opera sob um binário unificado desenvolvido em linguagem Rust⁵³. A sua eficiência na utilização da RAM e o seu modo embarcado local (in-memory ou via SQLite gerido diretamente através do pacote Python *qdrant-client*) erradicam qualquer sobrecarga orquestral, libertando a estação para focar exclusivamente na inferência⁵³.

Mais crítico ainda, a eficácia de uma recuperação orientada à manutenção não reside unicamente em vizinhança semântica. Se o nó diagnóstico estipula uma avaria na "Turbina B", o RAG não deve recuperar instruções relativas ao "Compressor Auxiliar" simplesmente porque os textos são semanticamente próximos. O Qdrant possui um motor de *Filtragem de Carga Útil* (Payload Filtering) nativamente enraizado no seu algoritmo de navegação em grafos, permitindo intersecções lógicas perfeitamente rápidas sem recorrer a verificações demoradas no pós-processamento⁵⁴.

Busca Híbrida e Parâmetros HNSW

A sintaxe de catálogos de peças exige precisão lexical e semântica. O Qdrant na estação local será configurado para processamento de Busca Híbrida, alojando tanto os embeddings densos (capturando significado) quanto vetores esparsos (modelos como SPLADE ou implementações do tipo BM25), ambos providenciados com extrema celeridade pelo CPU via FastEmbed²⁴. Na busca de vetores esparsos no Qdrant, as medições de distância baseiam-se inequivocamente no cálculo matemático do Produto Escalar (Dot Product) de índices invertidos e não na proximidade do co-seno, acelerando as intersecções de palavras exatas (como códigos serials)⁵⁸.

Para garantir máxima revocabilidade (recall) nas consultas, os hiperparâmetros de arquitetura subjacentes ao grafo HNSW (Hierarchical Navigable Small World) interno do Qdrant exigem ajuste fino e intencional. Dada a restrição mínima do tempo de ingestão dos manuais técnicos (uma operação executada maioritariamente *offline*), os limiares direcionam-se para a qualidade em detrimento da velocidade da compilação inicial do índice:

- **Parâmetro m:** Define o número de hiper-arestas por nó na topologia da grelha de conexões do HNSW. A recomendação em implementações industriais é elevar este valor para limites compreendidos entre 32 e 64. Grafos mais densos previnem a interrupção prematura das buscas por vizinhança, suportando consultas mais robustas à custa de aumentos toleráveis no volume de RAM⁵⁷.
- **Parâmetro ef_construct:** Define o alcance da exploração dos vizinhos durante a introdução de novos vetores na base de dados. Deve ser calibrado na faixa de 200 a

500, estendendo exponencialmente o limiar de qualidade da organização e consolidação do labirinto topológico do índice, com implicação mínima no tempo subsequente da procura propriamente dita⁵⁷.

- **Parâmetro `hnsw_ef`:** É configurado exclusivamente durante as consultas (tempo de execução). Operando num sistema suportado pelo vasto limite dos 32 GB de RAM, recomenda-se alargar a verificação dos nós adjacentes ajustando este valor para cima (ex.: 256), a fim de obter métricas de recuperação virtualmente perfeitas, garantindo que nenhum documento normativo seja esquecido⁵⁷.

Design de Sistemas e Comunicação de Backend

End-to-End

A arquitetura descrita consolida-se e orquestra-se em ambiente de produção através de um *backend* resiliente gerido pelo FastAPI. A adoção do FastAPI providencia a estrutura de E/S assíncrona orientada a eventos (ASGI) fundamental para coordenar a inferência do LLM (computacionalmente isolada e bloqueante na GPU) com o fluxo de interações do utilizador⁶¹.

Topologia de Exfiltração de Dados e Server-Sent Events (SSE)

O estrangulamento da experiência operacional num contexto de edge LLMs manifesta-se tipicamente no tempo necessário para que a primeira porção de texto útil chegue ao ecrã (Time to First Token - TTFT)¹³. A geração completa da resposta prescritiva por um LLM de 14B parâmetros a uma velocidade de 35 tokens/s pode facilmente ultrapassar a tolerância atencional de um operador sob pressão, forçando a espera mecânica de dezenas de segundos caso a comunicação se processe via requisições HTTP REST monolíticas típicas.

A mitigação técnica impõe a adoção de transmissões parciais, vulgarmente designadas de *streaming*. Embora os WebSockets forneçam canais de tráfego bidirecional simultâneo, a sua utilização neste cenário insere níveis imoderados de complexidade na orquestração dos estados e reconexões ativas do protocolo subjacente, adequando-se primordialmente a painéis colaborativos altamente dinâmicos⁶¹. Para o *chat* prescritivo, é implementada a estratégia de fluxo iterativo unidirecional utilizando os Protocolos de Eventos Enviados pelo Servidor (Server-Sent Events - SSE).

O SSE é nativamente amigável ao protocolo HTTP padrão, tolerando falhas de conexão transparentemente sem recorrer ao intercâmbio pesado (*handshake* sucessivo) que os WebSockets demandam e possuindo vasto suporte nos browsers⁶³. O fluxo SSE no FastAPI é instanciado combinando-se iteradores assíncronos em Python com classes empacotadoras do tipo `EventSourceResponse` provenientes de bibliotecas focadas (`fastapi.sse` ou `fastapi-sse`)⁶⁴. O controlador interno cede (através da diretriz de fluxo condicional `yield`) pacotes semânticos parciais, seguindo a estrutura padrão codificada do SSE:

HTTP

```
data: {"type": "content_block_delta", "delta": {"text": "A"}}
```

```
data: {"type": "content_block_delta", "delta": {"text": " anomalia"}}
```

```
data: {"type": "content_block_delta", "delta": {"text": " sugere"}}
```

Consequentemente, a comunicação flui do processo do servidor *background* governado pelo llama.cpp até à infraestrutura de controlo da Interface de Utilizador do *frontend* de forma incessante, fragmento a fragmento⁶³. Desta forma, a resposta prescritiva torna-se visível à medida que o raciocínio é processado pelo Qwen 2.5, proporcionando feedback em tempo real mesmo em condições onde a taxa efetiva de leitura se aproxime do limite prático do hardware na casa dos 20 a 30 tokens por segundo¹³.

Mapeamento do Fluxo Lógico (System Design)

A cadência de informação no sistema, desde a aquisição sensória até a saída no *chat* operacional, consolida-se assim:

1. **Ingestão de Dados (Sensor Ingestion):** O painel do operador remete via verbo HTTP POST um objeto JSON em série temporal.
2. **Validação Fronteiriça:** A diretiva Pydantic inerente ao FastAPI intercepta, digere e tipifica as variáveis do JSON. Assinaturas incongruentes (ex: Fator de Crista inferior a 1, impossível na física das ondas) são prontamente invalidadas, reportando código HTTP 422 para o cliente (Unprocessable Entity).
3. **Execução em Segundo Plano (Background Tasking):** Validados, os dados são introduzidos num invólucro do estado da orquestração do LangGraph. Para evitar o bloqueio (thread lock) principal do *loop* assíncrono do servidor web, a execução pesada de travessia do grafo e acionamento de hardware prossegue através do mapeamento num gestor de tarefas desacoplado no FastAPI.
4. **Processamento Neuro-Simbólico Integrado:** O LangGraph transita o estado: primeiramente acionando a matriz e calculando Mahalanobis via biblioteca nativa C no ambiente do Python, procedendo à rotação condicional para aquisição de contexto no Qdrant, e finalizando com a agregação do prompt a submeter à camada LLM.
5. **Exfiltração (Exfiltration):** A função cede (*yield*) contínua e ciclicamente os deltas resultantes ao canal de eventos, emergindo instantaneamente no ecrã do observador.

Proposta de Interface Visual (Frontend)

O painel visual e a interação humana atuam como o portal de aceitação desta elaborada malha operacional. Numa estação industrial de processamento de ponta, o *frontend* não tem a veleidade de atuar de forma desarticulada com as informações primárias que pautam a tomada de decisão. Ferramentas puramente construídas à base de *chatbots* clássicos obliteram o valor analítico subjacente, o que fomenta a desconfiança instintiva nos engenheiros no momento da intervenção crítica no maquinário.

A arquitetura visual deve fundir-se numa Aplicação de Página Única (Single Page Application -

SPA). Ambientes modulares, estruturados sobre frameworks reativos do ecossistema JavaScript (ex: React, orquestrado pela fluidez do Next.js) propiciam a melhor flexibilidade. Todavia, a utilização do Streamlit constitui uma via iterativa perfeitamente capacitada e extremamente comum para integrar ambientes nativos de engenharia Python. A filosofia do design do *layout* será suportada numa divisão ergonómica assintótica (Split-Pane Topology). O ecrã será particionado verticalmente sem sobreposições flutuantes:

1. Painel Esquerdo (Motor Simbólico / Painel Telemetria e Diagnóstico)

Reservando sensivelmente 40% a 45% do panorama visual, este segmento acolhe o rigor estático das estatísticas vitais e a transparência matemática. A interface incluirá:

- **Dashboards Qualitativos ISO 20816-3:** O foco imediato do olhar recairá num sistema de medição simplificado. Dependendo do valor bruto do RMS (em mm/s) cruzado com o tipo de estrutura mecânica rígida ou flexível e potência do maquinário, o gráfico será destacado num limite de severidade com cores normativas exatas². As métricas estáticas do Fator de Crista e da Curtose flutuarão junto deste painel como referências auxiliares vitais no despiste rápido dos rolamentos³⁴.
- **Resultados de Similaridade de Mahalanobis:** A taxonomia matemática isolada no banner.csv será descrita não pelo chat, mas através de crachás estáticos: (Exemplo: "Diagnóstico Primário Identificado: *Defeito Moderado em Pista Externa* | Grau de Confiança Matemático: 94%"). A explicitação inegável destes dados forja o elemento central da confiança operacional, provando visualmente ao operador que o enquadramento fundamental de rutura adveio de um algoritmo determinístico e imutável.

2. Painel Direito (Motor Neural / Chat Prescritivo Interativo)

Alocando o restante escopo visual, situar-se-á a componente de comunicação interativa orientada pelo motor LLM. Quando acionado por um evento ou consulta intencional, esta região receberá os feixes de fluxo transmitidos assincronamente pelo protocolo SSE, preenchendo as prescrições de intervenção a velocidade natural.

As salvaguardas de credibilidade integradas através da componente RAG não devem permanecer obscuras. O *layout* preverá nativamente:

- **Distintivos de Citação (Citation Badges):** Cada resposta injetada e elaborada por parte da inteligência artificial sobre torques de aperto de parafusos, volumes e tipos de purgas de lubrificação, ou medidas corretivas associadas à falha previamente identificada será pontuada com ícones clicáveis⁴. Ao clicar, componentes adicionais sobrepõem os recortes precisos dos textos recuperados pelo motor Qdrant, estabelecendo hiperligações diretas ou ancoragens na página correta do manual oficial da fábrica.
- **Fallbacks para Anomalias Inéditas:** Em cumprimento com a aresta condicional implementada no LangGraph, caso a distanciometria do Mahalanobis rejeite o padrão de vibração em favor do estatuto Fora de Distribuição (OOD)⁵, o LLM manifestar-se-á unicamente declarando a desconformidade isolada. Ao processar este estado invulgar, o painel do React ou do Streamlit reagirá, ocultando dinamicamente a área normal de

interrogação de manuais e ativando condicionalmente um modal ou ecrã de preenchimento (Formulário de Relato). Este incentiva expressamente a chefia operacional e o técnico no solo a validarem os sintomas fisicamente e a introduzirem-nos num diário de anomalias primárias (Engineering Ticket). O que culmina por retroalimentar a base histórica banner.csv e solicitar indiretamente uma submissão posterior de novos procedimentos à base de dados de vetor de conhecimento normativo, garantindo o ciclo orgânico da otimização industrial orientada ao dado real e ao empirismo.

Conclusão

A materialização da orquestração estrita de Manutenção Prescritiva num enclave de silício local dotado das barreiras singelas dos 16 GB de VRAM assinala uma evolução notável no panorama industrial de IA em borda. O desígnio arquitetural não é alcançado forçando modelos não alinhados à realização de cálculos físicos, mas por abraçar corajosamente a sua vocação linguística dentro dos trâmites perfeitamente coordenados e vigiados por funções algébricas severas.

A amálgama e convergência metodológica exposta no presente documento prescreve a implementação conjunta do rigor métrico e estatístico da distância de Mahalanobis enraizado na normativa ISO 20816-3; a robustez dos cálculos probabilísticos efetuados à margem dos desvios alucinatórios orquestrados pelo cérebro dos grafos LangGraph; as virtudes hiper-condensadas, multilingues e flexíveis do modelo estruturado da categoria dos 14B parâmetros (Qwen 2.5), embutidos num sistema de memória quantizada pela eficiência minimalista mas colossal e rigorosa de llama.cpp; tudo alicerçado por um repositório leve de alta afinidade e rápida destilação da família FastEmbed e Qdrant.

Ao separar física rigorosa da síntese de raciocínio lógico em micro-componentes especializados que comunicam agilmente através do paralelismo robusto do FastAPI (SS Events), os tomadores de decisão corporativa mitigam dramaticamente as latências incontroláveis de rede das clouds públicas e erigem internamente defesas estruturais insondáveis a anomalias de interpretação cega de algoritmos neurais. Esta topologia atinge, no final, a promessa do sistema que garante segurança industrial plena perante o desconhecido, e velocidade sem rodeios para diagnosticar o previsto.

Referências citadas

1. Deploy Qwen 2.5 72B on GPU Cloud: VRAM Sizing and vLLM Setup - Lyceum Technology,
<https://lyceum.technology/magazine/deploy-qwen-2-5-72b-gpu-cloud/>
2. ISO 20816-3: Comprehensive Guide Vibrations in Rotating Machines - DSP Analytic,
<https://dspanalytic.com/en/sin-categorizar/iso-20816-3-comprehensive-guide-for-the-evaluation-of-vibration-on-rotating-machinery/>
3. Physics-Informed Multimodal Bearing Fault Classification under Variable Operating Conditions using Transfer Learning - arXiv,

- <https://arxiv.org/html/2508.07536v1>
4. Reducing Hallucinations Using Prompt Engineering and RAG - DZone, <https://dzone.com/articles/reducing-llm-hallucinations-prompt-engineering-and-rag>
 5. Mahalanobis++: Improving OOD Detection via Feature Normalization - arXiv, <https://arxiv.org/html/2505.18032v1>
 6. [2204.06507] Out-of-Distribution Detection with Deep Nearest Neighbors - arXiv, <https://arxiv.org/abs/2204.06507>
 7. Best LLMs for 12 GB VRAM: RTX 5070 Model Guide 2026 - GIGAGPU, <https://gigagpu.com/best-llm-for-12gb-vram/>
 8. Best Qwen Model for 16GB VRAM (2026): Which One to Run | Local AI Master, <https://localaimaster.com/vram/best-qwen-model-16gb-vram>
 9. What's the best models available today to run on systems with 8 GB / 16 GB / 24 GB / 48 GB / 72 GB / 96 GB of VRAM today? : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1k4av1q/whats_the_best_models_available_today_to_run_on/
 10. Llama 3.1 8B Instruct vs Qwen2.5 14B Instruct: Benchmarks, Pricing & Which Is Better in 2026 - LLM Stats, <https://llm-stats.com/models/compare/llama-3.1-8b-instruct-vs-qwen-2.5-14b-instruct>
 11. How to Use Llama 3 Locally: An Engineer's Guide | QWE AI Academy, <https://www.qwe.edu.pl/tutorial/llama-3-locally-engineer-guide/>
 12. Qwen2.5: A Party of Foundation Models, <https://qwen.ai/blog?id=qwen2.5>
 13. Small LLM Performance Benchmark - Research Report - AscentCore, <https://ascentcore.com/2026/04/01/small-llm-performance-benchmark/>
 14. llama.cpp - how to free up even more space on your GPU : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1u8i79d/llamacpp_how_to_free_up_even_more_space_on_your/
 15. Llama.cpp vs vLLM: Which Local LLM Engine Actually Scales? - YouTube, <https://www.youtube.com/watch?v=0ujh7hfutq0>
 16. llama-server(1) — llama.cpp-tools — Debian unstable - Debian Manpages, <https://manpages.debian.org/unstable/llama.cpp-tools/llama-server.1.en.html>
 17. vLLM vs llama.cpp: Diferenças de Eficiência em Contextos Grandes no Qwen3.5-4B AWQ, https://www.reddit.com/r/LocalLLaMA/comments/1sfnjoh/vllm_vs_llamacpp_huge_context_efficiency/?tl=pt-br
 18. llama.cpp oom issue : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1tn72x3/llamacpp_oom_issue/
 19. [User] Memory usage is extremely low when running 65b 4-bit models. (Only use 5GB) · Issue #864 · ggml-org/llama.cpp - GitHub, <https://github.com/ggml-org/llama.cpp/issues/864>
 20. FastEmbed: Qdrant's Efficient Python Library for Embedding Generation, <https://qdrant.tech/articles/fastembed/>
 21. GitHub - qdrant/fastembed: Fast, Accurate, Lightweight Python library to make State of the Art Embedding, <https://github.com/qdrant/fastembed>

22. skatzR/USER-BGE-M3-ONNX-INT8 - Hugging Face,
<https://huggingface.co/skatzR/USER-BGE-M3-ONNX-INT8>
23. Build local AI Embeddings with FastEmbed | by Sanidhya Goel | Medium,
<https://medium.com/@wriath18/build-local-ai-embeddings-with-fastembed-787ac266f4ec>
24. FastEmbed - Qdrant, <https://qdrant.tech/documentation/fastembed/>
25. Embedding Models Compared: What Actually Matters for RAG | by Ali Imran - Medium,
<https://medium.com/@saliimranz12/embedding-models-compared-what-actually-matters-for-rag-f17881893901>
26. fastembed 5.17.3 - Docs.rs, <https://docs.rs/crate/fastembed/latest>
27. Optimize Throughput - FastEmbed - Qdrant,
<https://qdrant.tech/documentation/fastembed/fastembed-optimize/>
28. High memory usage when embedding large texts · Issue #222 · qdrant/fastembed - GitHub, <https://github.com/qdrant/fastembed/issues/222>
29. ISO 20816-3: Vibration Limits for Industrial Machines,
<https://vibromera.eu/glossary/iso-20816-3/>
30. ISO 10816-3 Vibration Severity: Zones, Limits and How to Read Them - Fabrico CMMS, <https://www.fabrico.io/blog/iso-10816-3-vibration-severity/>
31. Vibration Analysis Basics: How to Read Machine Vibration (2026 Guide),
<https://plcprogramming.io/blog/vibration-analysis-basics>
32. INTERNATIONAL STANDARD ISO 20816-1,
<https://cdn.standards.iteh.ai/samples/63180/ace1232a76714dc48087a6cfe49c2bda/ISO-20816-1-2016.pdf>
33. vibration analysis for early detection of bearing failures gam kheng shiang universiti tunku abdul rahman,
http://eprints.utar.edu.my/7394/1/ME_2100685_FYP_Report_-_KHENG_SHIANG_GAM.pdf
34. Vibration Energy at Damage-Based Statistical Approach to Detect Multiple Damages in Roller Bearings - MDPI, <https://www.mdpi.com/2076-3417/12/17/8541>
35. Spectrum Analysis - SKF,
https://cdn.skfmediahub.skf.com/api/public/0901d1968024acef/pdf_preview_medium/0901d1968024acef_pdf_preview_medium.pdf
36. Bearing Fault Detection Vibration Analysis - How To Measure Vibration Frequency - NCD.io, <https://ncd.io/blog/bearing-fault-detection-vibration-analysis/>
37. [2509.21219] An enhanced statistical feature fusion approach using an improved distance evaluation algorithm and weighted K-nearest neighbor for bearing fault diagnosis - arXiv, <https://arxiv.org/abs/2509.21219>
38. 20th Iranian International Industrial Engineering Conference (IIIEC 2025) An enhanced statistical feature fusion approach using - arXiv,
<https://arxiv.org/pdf/2509.21219>
39. mahalanobis — SciPy v1.18.0 Manual,
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.mahalanobis.html>
40. Mahalanobis Distance - Understanding the math with examples (python),

- <https://machinelearningplus.com/statistics/mahalanobis-distance/>
41. Apparently issue on mahalanobis distance calculation while using pdist #8801 - GitHub, <https://github.com/scipy/scipy/issues/8801>
 42. A Novel k-Nearest Neighbor Method with Distribution Discrepancy and Differential Feature Importance for Rolling Bearing Fault Diagnosis - PubMed, <https://pubmed.ncbi.nlm.nih.gov/42329886/>
 43. LangGraph - LangChain, <https://www.langchain.com/blog/langgraph>
 44. Graph API overview - Docs by LangChain, <https://docs.langchain.com/oss/python/langgraph/graph-api>
 45. Learning LangGraph.js Part 2: Conditional Edges | by Guy Royse | The Guy Wire - Medium, <https://medium.com/the-guy-wire/learning-langgraph-js-part-2-conditional-edges-4672c35ff42f>
 46. LangGraph Best Practices - Swarnendu De, <https://www.swarnendu.de/blog/langgraph-best-practices/>
 47. add_conditional_edges | langgraph - LangChain Reference, https://reference.langchain.com/python/langgraph/graph/state/StateGraph/add_conditional_edges
 48. LangGraph Basics: Part 3 — Conditional Edges & Routing Logic - Shafiqul AI, https://shafiqulai.github.io/blogs/blog_10.html
 49. A Beginner's Guide to Getting Started on Edges in LangGraph | by Damilola Oyedunmade | AI Engineering BootCamp | Medium, <https://medium.com/ai-engineering-bootcamp/a-beginners-guide-to-getting-started-on-edges-in-langgraph-1da10bdc5623>
 50. ISO 20816-3:2022 Machine Vibration Standards | PDF - Scribd, <https://www.scribd.com/document/929819363/ISO-20816-3-2022-ISO-20816-3-2022-ISO-20816-3-2022>
 51. Document - PyMuPDF - Read the Docs, <https://pymupdf.readthedocs.io/en/latest/document.html>
 52. RAG (Retrieval-Augmented Generation): The Full Technical Architecture Guide, <https://eph4.ai/rag-systems/full-rag-technical-architecture-master-guide/>
 53. Milvus vs Qdrant 2026: Pick the Right Vector DB - Kunal Ganglani, <https://www.kunalganglani.com/blog/milvus-vs-qdrant>
 54. Qdrant vs Milvus: Detailed Comparison - IngestIQ, <https://ingestiq.ai/resources/comparisons/qdrant-vs-milvus>
 55. Milvus vs Qdrant: Which Open-Source Vector Database? (2026) - Layer3Labs, <https://www.layer3labs.io/comparisons/milvus-vs-qdrant>
 56. Qdrant Vector Database: Production Tutorial with Python Code (2026) - Cohorte, <https://cohorte.co/blog/a-developers-friendly-guide-to-qdrant-vector-database>
 57. HNSW Indexing Fundamentals - Qdrant, <https://qdrant.tech/course/essentials/day-2/what-is-hnsw/>
 58. Sparse Vectors and Inverted Indexes - Qdrant, <https://qdrant.tech/course/essentials/day-3/sparse-vectors/>
 59. Demo: Keyword Search with Sparse Vectors - Qdrant, <https://qdrant.tech/course/essentials/day-3/sparse-retrieval-demo/>

60. Indexing - Qdrant, <https://qdrant.tech/documentation/manage-data/indexing/>
61. Implementing Server-Sent Events (SSE) with FastAPI: Real-Time Updates Made Simple,
<https://mahdijafaridev.medium.com/implementing-server-sent-events-sse-with-fastapi-real-time-updates-made-simple-6492f8bfc154>
62. Streaming local LLM with FastAPI, Llama.cpp and Langchain - Stack Overflow,
<https://stackoverflow.com/questions/77867894/streaming-local-llm-with-fastapi-llama-cpp-and-langchain>
63. Server-Sent Events for LLM Streaming | Learnixio,
<https://learnixio.io/blog/fapi-sse-streaming>
64. Server-Sent Events (SSE) - FastAPI,
<https://fastapi.tiangolo.com/tutorial/server-sent-events/>
65. fastapi-sse - PyPI, <https://pypi.org/project/fastapi-sse/>
66. LLM Streaming Responses with FastAPI and Anthropic SDK: SSE in Production,
<https://devopsboys.com/blog/llm-streaming-fastapi-anthropic-sdk-2026>
67. Benchmarked Qwen 3 8B, Llama 3.1 8B & Qwen 2.5 7B vs GPT-4o on M4 Mac mini. Honest results : r/ollama - Reddit,
https://www.reddit.com/r/ollama/comments/1tgsz66/benchmarked_qwen_3_8b_llama_3_1_8b_qwen_2_5_7b_vs/